1
Getting Started

We kick off this book with a collection of relatively elementary programming problems, none of which require ideas more advanced than arrays and iteration.

Elementary does not necessarily mean easy, however! These problems provide a good introduction to the demanding nature of the robot judge, and the need to read carefully and understand specifications. They also provide an opportunity to discuss programming styles best suited to getting the job done.

To help you get started, we begin with a description of the robot judges and their idiosyncrasies. We follow with a discussion of basic programming style and data structures before introducing our first set of problems. As in all chapters in this book, we follow with hints for selected problems and notes for further study.

1.1 Getting Started With the Judge

This book is designed to be used in tandem with one (or both) of two robot judging websites. The Programming Challenges judge *http://www.programming-challenges.com* has been set up specifically to help you get the most from the challenges in this book. The Universidad de Valladolid judge *http://online-judge.uva.es* has a different interface as well as hundreds of additional problems available.

All the problems in the book can be judged from either website, which are both administered by Miguel Revilla. In this section, we describe how to use the two judges and explain the differences between them. Be aware that both sites are living, breathing projects, so these procedures may evolve over time. Check the current instructions at each site for clarification.

Your first task is to get an account for the judge of your choice. You will be asked to give a password governing access to your personal data, specifically your name and your email address. If you forget your password, clicking the appropriate button will get it emailed back to you.

Note that the contestant rosters of the two sites are currently kept distinct, but there is no reason why you should not register for both of them and enjoy their distinct advantages.

1.1.1 The Programming Challenges Robot Judge

The Programming Challenges website (*http://www.programming-challenges.com*) provides special features associated with each of the problems in this book. For example, a description of each challenge appearing in the book is given on site, along with down-loadable input and output files to eliminate the need for you to type this test data.

The Programming Challenges site uses a web interface for submission (the *Submit-o-Matic*) instead of the email interface of the UVa judge. This makes submission much easier and more reliable, and provides for quicker response.

Each problem in this book has two associated ID numbers, one for each judge. One advantage of the web interface is that the identifier for the Programming Challenges site (the PC ID) is not necessary for most submissions. The problem descriptions in this book have been rewritten for clarity; thus they often differ from the descriptions on the UVa judge in minor ways. However, the problems they describe are identical. Thus any solution scored as correct on one judge *should be* scored correct on the other as well.

The Programming Challenges site has a special course management interface, which permits an instructor to maintain a roster of students in each class and see their submissions and results. It also contains a program similarity tester so the instructor can verify that the solutions each student submits are indeed his or her own work. This makes it "bad karma" to hunt for solutions on the web or in your classmate's directories.

1.1.2 The Universidad de Valladolid Robot Judge

All of the problems in this book and many more appear on the Universidad de Valladolid robot judge *http://online-judge.uva.es*, the largest collection of programming problems in the world. We encourage anyone whose appetite has been whetted by our challenges to continue their studies there.

After registering on the UVa judge, you will receive email containing an ID number which will uniquely identify your programs to the judge. You will need this ID number for every solution you submit.

The UVa judge is gradually adopting a web interface but currently uses email submission. Solutions are emailed directly to *judge@uva.es* after being annotated with enough information to tell the judge which problem you are trying to solve, who the author is, and what programming language you are using.

Specifically, each submitted program must contain a line (at any location) with an `@JUDGE_ID:` field. Usually, this line is placed inside a comment. For example,

```
/*   @JUDGE_ID:   1000AA   100   C   "Dynamic Programming"   */
```

The argument after the `@JUDGE_ID:` is your user ID (`1000AA` in the example). This is followed by the problem number (100 in the example), and then by the language used. *Make sure you use the UVa ID number for all submissions to this judge!* Upper- and lowercase letters are indistinguishable. If you fail to specify the programming language, the judge will try to auto-detect it – but why play games? Finally, if you have used any interesting algorithm or method, you may include a note to that effect between quotes, such as `Dynamic Programming` in the example above.

Bracketing your program with beginning/end of source comments is a good way to make sure the judge is not confused by junk appended by your mailer.

```
/* @BEGIN_OF_SOURCE_CODE */

your program here

/* @END_OF_SOURCE_CODE */
```

Certain mysterious errors will go away when you do this.

1.1.3 Feedback From the Judge

Students should be aware that both judges are often very picky as to what denotes a correct solution. It is very important to interpret the problem specifications properly. *Never* make an assumption which is not explicitly stated in the specs. There is absolutely no reason to believe that the input is sorted, the graphs are connected, or that the integers used in a problem are positive and reasonably small unless it says so in the specification.

Just like the human judges of the ACM International Collegiate Programming Contest, the online judge provides you with very little feedback about what is wrong with your program. The judge is likely to return one of the following verdicts:

- *Accepted (AC)* — Congratulations! Your program is correct, and runs within the given time and memory limits.
- *Presentation Error (PE)* — Your program outputs are correct but are not presented in the specified format. Check for spaces, left/right justification, line feeds, etc.
- *Accepted (PE)* — Your program has a minor presentation error, but the judge is letting you off with a warning. Don't be concerned, because many problems have somewhat ambiguous output specifications. Usually your problems are something as trivial as an extra blank at the end of a line, so stop here and declare victory.

- *Wrong Answer (WA)* — This you should concern you, because your program returned an incorrect answer to one or more of the judge's secret test cases. You have some more debugging to do.
- *Compile Error (CE)* — The compiler could not figure out how to compile your program. The resulting compiler messages will be returned to you. Warning messages that do not interfere with compilation are ignored by the judge.
- *Runtime Error (RE)* — Your program failed during execution due to a segmentation fault, floating point exception, or similar problem. Its dying message will be sent back to you. Check for invalid pointer references or division by zero.
- *Time Limit Exceeded (TL)* — Your program took too much time on at least one of the test cases, so you likely have a problem with efficiency. Just because you ran out of time on one input does not mean you were correct on all the others, however!
- *Memory Limit Exceeded (ML)* — Your program tried to use more memory than the judge's default settings.
- *Output Limit Exceeded (OL)* — Your program tried to print too much output. This usually means it is trapped in a infinite loop.
- *Restricted Function (RF)* — Your source program tried to use an illegal system function such as `fork()` or `fopen()`. Behave yourself.
- *Submission Error (SE)* — You did not correctly specify one or more of the information fields, perhaps giving an incorrect user ID or problem number.

Just to reiterate: if your program is found guilty of having a wrong answer, the judge will not show you which test case it failed on, or give you any additional hints as to why it failed. This is why it is so essential to review the specifications carefully. Even when you may be *sure* that your program is correct, the judge may keep saying no. Perhaps you are overlooking a boundary case or assuming something which just ain't so. Resubmitting the program without change does you absolutely no good. Read the problem again to make sure it says what you thought it did.

The judge occasionally returns a more exotic verdict which is essentially independent of your solution. See the appropriate website for details.

1.2 Choosing Your Weapon

What programming language should you use in your battles with the judge? Most likely, the language which you know best. The judge currently accepts programs written in C, C++, Pascal, and Java, so your favorite language is probably available. One programming language may well be better than another for a specific programming task. However, these problems test general problem-solving skills far more than portability, modularity, or efficiency, which are the usual dimensions by which languages are compared.

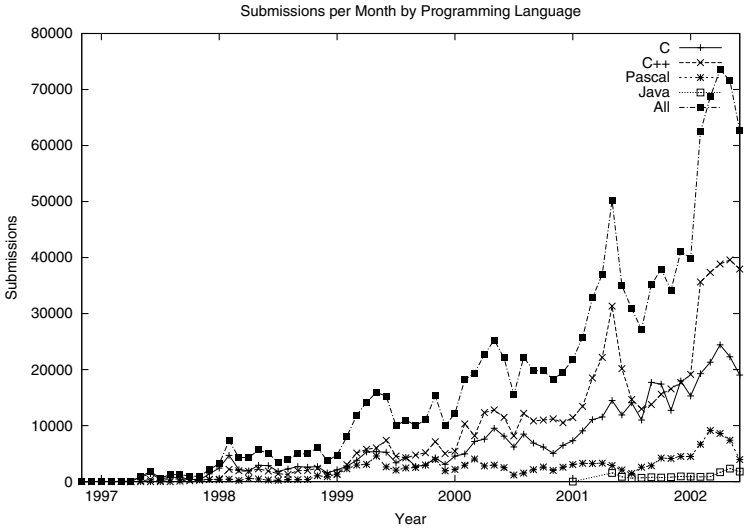


Figure 1.1. Robot Judge Submissions by Programming Language Through December 2002.

1.2.1 Programming Languages

The four languages supported by the judge were designed at different times with different goals in mind:

- *Pascal* — The most popular educational programming language of the 1980s, Pascal was designed to encourage good structured-programming habits. Its popularity has eroded almost to the point of extinction, but it retains a foothold in high schools and in Eastern Europe.

- *C* — The original language of the UNIX operating system, C was designed to provide experienced programmers with the power to do whatever needs to be done. This includes the power to hang yourself by invalid pointer references and invalid type casting. Developments in object-oriented programming during the 1990s lead to the new and improved...

- *C++* — The first commercially successful object-oriented language pulled off the neat trick of maintaining backward compatibility with C while incorporating new data abstraction and inheritance mechanisms. C++ became the primary programming language for teaching and industry during the mid-to-late 1990s, but now it looks over its shoulder at...

- *Java* — Designed as a language to support mobile programs, Java has special security mechanisms to avoid common programmer errors such as array out-of-bounds violations and illegal pointer access. It is a full-featured programming language which can do everything the others can and more.

Lang	Total	AC	PE	WA	CE	RE	TL	ML	OL	RF
C	451447	31.9%	6.7%	35.4%	8.6%	9.1%	6.2%	0.4%	1.1%	0.6%
C++	639565	28.9%	6.3%	36.8%	9.6%	9.0%	7.1%	0.6%	1.0%	0.7%
Java	16373	17.9%	3.6%	36.2%	29.8%	0.5%	8.5%	1.0%	0.5%	2.0%
Pascal	149408	27.8%	5.5%	41.8%	10.1%	6.2%	7.2%	0.4%	0.4%	0.5%
All	1256793	29.7%	6.3%	36.9%	9.6%	8.6%	6.8%	0.5%	1.0%	0.6%

Table 1.1. The Judge's Verdicts by Programming Language (Through December 2002).

Note that each of the judge's programming languages have compiler and system-specific idiosyncrasies. Thus a program which runs on your machine may not run on the judge. Read the language notes on your judge's website carefully to minimize trouble, particularly if you are using Java.

It is interesting to look at which languages people have been using. As of December 2002 over 1,250,000 program submissions have been sent to the robot judge. Almost half of them were in C++, with almost another third in C. Only a tiny fraction were written in Java, but that is not a fair test since the judge did not accept Java programs until November 2001.

These submissions are broken down by month in Figure 1.1. C proved the most popular language until late 1999, when C++ surged ahead. It is interesting to note the annual spike in demand each fall as students train for the ACM International Collegiate Programming Contest regional competitions. Every year the judge gets busier, as more and more students seek trial in its court.

It is also interesting to look at the judge's verdicts by programming language. These are tabulated in Table 1.1, according to the response codes described in Section 1.1.3. The verdicts are quite consistent across the board. However, the frequencies of certain types of errors appear to be language dependent. C++ programs run out of time and memory more often than C language programs, a sign that C++ is a relative resource hog. C has a slightly higher acceptance rate than C++, presumably reflecting its popularity at an earlier state in the judge's development. Pascal has the lowest rate of restricted function errors, reflecting its origins as a nice, safe language for students to play with. Java has had *far* more than its share of compiler errors to date, but it also crashes much less often than the other languages. Safety is indeed a virtue.

But the basic lesson is that the tools do not make the man. Your language doesn't solve the problems – you do.

1.2.2 Reading Our Programs

Several programming examples appear in this book, illustrating programming techniques and providing complete implementations of fundamental algorithms. All of this code is available at *http://www.programming-challenges.com* for you to use and experiment with. There is no finer way to debug programs than having them read by several thousand bright students, so look there for errata and revised solutions.

Our programming examples are implemented in a vanilla subset of C, which we hope will be understandable by all of our readers with relatively little effort. C itself is a

subset of C++ and its syntax is quite similar to Java. We have taken care to avoid using weird C-specific constructs, pointer structures and dynamic memory allocation throughout this book, so what remain should be familiar to users of all four of the judge's programming languages.

We provide a few hints about C below which may be helpful in reading our programs:

- *Parameter Passing* — All parameters in C are passed by call-by-value, meaning that copies of all arguments are made on function calls. This would *seem* to suggest that it is impossible to write functions that have side effects. Instead, C encourages you to pass a pointer to any argument that you intend to modify within the body of the function.

  Our only use of pointers will be in parameter passing. The pointer to `x` is denoted by `&x`, while the item pointed to by `p` is denoted `*p`. Do not get confused between multiplication and pointer dereferencing!

- *Data Types* — C supports several basic data types, including `int`, `float`, and `char`, which should all be self-explanatory. Higher precision `int`s and `float`s are denoted `long` and `double`, respectively. All functions return a value of type `int` if not otherwise specified.

- *Arrays* — C array indices always range from 0 to $n-1$, where n is the number of elements in the array. Thus if we want to start with a first index of 1 for convenience, we had better remember to allocate room for $n+1$ elements in the array. No runtime checking is performed on the validity of array bounds, so such errors are a common cause of program crashes.

  We are not always consistent as to where the first element of each array is located. Starting from 0 is the traditional C style. However, it is sometimes clearer or easier to start at 1, and we are willing to pay one extra memory location for the privilege. Try not to be confused when reading our code.

- *Operators* — C contains a few essential operators which may be mysterious to some readers. The integer remainder or *mod* operation is denoted `%`. The logical-and and logical-or operations which appear in conditional statements are denoted `&&` and `||`, respectively.

1.2.3 Standard Input/Output

UNIX programmers are familiar with notions of filters and pipes–programs that take one stream of input and produce one stream of output. The output of such programs is suitable to feed to other programs as input. The paradigm is one of stringing lots of little programs together rather than producing big, complicated software systems that try to do everything.

This software tools philosophy has taken somewhat of a beating in recent years due to the popularity of graphical user interfaces. Many programmers instinctively put a point-and-click interface on every program. But such interfaces can make it very difficult to transfer data from one program to another. It is easy to manipulate text output in another program, but what can you do with an image other than look at it?

```c
#include<stdio.h>

int main() {
   long p,q,r;

   while (scanf("%ld %ld",&p,&q)
               !=EOF) {
     if (q>p) r=q-p;
     else r=p-q;

     printf("%ld\n",r);
   }
}
```

```cpp
#include<iostream.h>

void main()
{
long long a,b,c;

while (cin>>a>>b) {
   if (b>a)
      c=b-a;
   else
      c=a-b;
   cout << c << endl;
}
}
```

```
{$N+}
program acm;
var
  a, b, c : integer;
begin
  while not eof do
  begin
    readln(a, b);
    if b > a then
    begin
      c := b;
      b := a;
      a := c
    end;
    writeln(a - b);
  end
end.
```

Figure 1.2. Standard Input/Output Examples in C (left), C++ (center), and Pascal (right).

The judge's I/O standards reflect the official ACM programming contest rules. Each program must read the test data from the standard input and print the results to the standard output. Programs are not allowed to open files or to execute certain system calls.

Standard input/output is fairly easy in C, C++, and Pascal. Figure 1.2 provides a simple example in each language that reads in two numbers per line and reports the absolute value of their difference. Note how your favorite language tests for the end-of-file terminating condition. Most problems make input processing even easier by specifying a count of the number of examples or describing a special termination line.

Most languages provide powerful formatted I/O functions. When used properly, single-line commands can render unnecessary certain painful parsing and formatting routines written by those who didn't read the manual.

Standard input/output is *not* easy in Java, however. A electronic template for Java I/O (35 lines long) is available from *http://www.programming-challenges.com*. Set it up once and use it for all your entries.

Java programs submitted to the judge must consist of a single source code file. They are currently compiled and run as native applications using the `gcj` compiler, although this may change in the future. Note that `java::io` use is restricted; which implies that some features are not available. Network functions and threads are also unavailable. However, methods from `math`, `util` and other common packages are authorized. All programs must begin in a static main method in a `Main` class. Do not use public classes: even `Main` must be non-public to avoid compile errors. However, you can add and instance as many classes as needed.

If you find yourself using an operating system/compiler which makes it difficult to use standard input/output, note that the judge always defines the `ONLINE_JUDGE` symbol while compiling your program. Thus, you can test for it and redirect the input/output to a file when running on your own system.

1.3 Programming Hints

It is not our purpose in this book to teach you how to program, only how to program better. We assume you are familiar with such fundamental concepts as variables, conditional statements (e.g., `if-then-else`, `case`), iteration primitives (e.g., `for-do`, `while-do`, `repeat-until`), subroutines, and functions. If you are unfamiliar with these concepts you may have picked up the wrong book, but buy it anyway for later use.

It is important to realize how much power there is in what you already know. In principle, every interesting algorithm/program can be built from what you learn in a first programming course. The powerful features of modern programming languages are not really necessary to build interesting things – only to do them in cleaner, better ways.

To put it another way, one becomes a good writer not by learning additional vocabulary words but by finding something to say. After one or two programming courses, you know all the words you need to make yourself understood. The problems in this book strive to give you something interesting to say.

We offer a few low-level coding hints that are helpful in building quality programs. The bad examples all come from actual submissions to the robot judge.

- *Write the Comments First* — Start your programs and procedures by writing a few sentences explaining what they are supposed to do. This is important because if you can't *easily* write these comments, you probably don't really understand what the program does. We find it much easier to debug our comments than our programs, and believe the additional typing is time very well spent. Of course, with the time pressure of a contest comes a tendency to get sloppy, but do so at your own risk.

- *Document Each Variable* — Write a one-line comment for each variable when you declare it so you know what it does. Again, if you can't describe it easily, you don't know why it is there. You will likely be living with the program for at least a few debug cycles, and this is a modest investment in readability which you will come to appreciate.

- *Use Symbolic Constants* — Whenever you have a constant in your program (input size, mathematical constant, data structure size, etc.) declare it to be so at the top of your program. Horribly insidious errors can result from using inconsistent constants in a program. Of course, the symbolic name helps only if you actually use it in your program whenever you need the constant...

- *Use Enumerated Types for a Reason* — Enumerated types (i.e., symbolic variables such Booleans (`true`,`false`)) can be terrific aids to understanding. However, they are often unnecessary in short programs. Note this example representing the suit (club, diamond, heart, spade) of a deck of cards:

```
switch(cursuit) {
    case 'C':
      newcard.suit = C;
```

```
      break;
    case 'D':
      newcard.suit = D;
      break;
    case 'H':
      newcard.suit = H;
      break;
    case 'S':
      newcard.suit = S;
...
```

No additional clarity arises from using the enumerated variables `(C,D,H,S)` over the original character representation `('C','D','H','S')`, only additional opportunities for error.

- *Use Subroutines To Avoid Redundant Code* — Read the following program fragment managing the state of a rectangular board, and think how you might shorten and simplify it:

```
...

     while (c != '0') {
        scanf("%c", &c);
        if (c == 'A') {
            if (row-1 >= 0) {
                temp = b[row-1][col];
                b[row-1][col] = ' ';
                b[row][col] = temp;
                row = row-1;
            }
        }
        else if (c == 'B') {
            if (row+1 <= BOARDSIZE-1) {
                temp = b[row+1][col];
                b[row+1][col] = ' ';
                b[row][col] = temp;
                row = row+1;
            }
        }
...
```

In the full program, there were four blocks of three lines each moving a value to a neighboring cell. Mistyping a single + or − would have lethal consequences. Much safer would be to write a single move-swap routine and call it with the proper arguments.

- *Make Your Debugging Statements Meaningful* — Learn to use the debugging environment on your system. This will enable you to stop execution at a given statement or condition, so you can see what the values of all associated variables are. This is usually faster and easier than typing in a bunch of print statements. But if you *are* going to insert debugging print statements, make them say something. Print out all relevant variables, and *label* the printed quantity with the variable name. Otherwise it is easy to get lost in your own debugging output.

Most computer science students are now well-versed in *object-oriented programming*, a software engineering philosophy designed to construct reusable software components and exploit them. Object-oriented programming is useful to build large, reusable programs. However, most of the programming challenge problems in this book are designed to be solved by short, clever programs. The basic assumption of object-oriented programming just does not apply in this domain, so defining complicated new objects (as opposed to *using* predefined objects) is likely to be a waste of time.

The trick to successful programming is not abandoning style, but using one appropriate to the scale of the job.

1.4 Elementary Data Types

Simple data structures such as arrays have an important advantage over more sophisticated data structures such as linked lists: they are simple. Many kinds of errors in pointer-based structures simply cannot happen with static arrays.

The sign of a mature professional is keeping the simple jobs simple. This is particularly challenging for those who are just learning a new subject. Medical students provide a classic example of this problem. After sitting through a few lectures on obscure tropical diseases, a young doctor worries that any patient with a sniffle and a rash might have the Ebola virus or bubonic plague, while a more experienced physician just sends them home with a bottle of aspirin.

Likewise, you may have recently learned about balanced binary search trees, exception handling, parallel processing, and various models of object inheritance. These are all useful and important subjects. But they are not necessarily the best way to get a correct program working for a simple job.

So, yes, pointer-based structures are very powerful if you do not know the maximum size of the problem in advance, or in supporting fast search and update operations. However, many of the problems you will be solving here have maximum sizes specified. Further, the robot judge typically allows several seconds for your job to complete, which is a lot of computation time when you stop to think about it. You don't get extra points for finishing faster.

So what is the simple, mature approach to data structures? First, be familiar with the basic primitive data *types* built into your programming language. In principle, you can build just about anything you want from just these:

- *Arrays* — This workhorse data type permits access to data by position, not content, just like the house numbers on a street permit access by address, not

name. They are used to store sequences of single-type elements such as integers, reals, or compound objects such as records. Arrays of characters can be used to represent text strings, while arrays of text strings can be used to represent just about anything.

Sentinels can be a useful technique to simplify array-based programming. A sentinel is a guard element, implicitly checking that the program does not run beyond the bounds of the array without performing an explicit test. Consider the case of inserting element x into the proper position among n elements in a sorted array a. We can explicitly test each step to see whether we have hit the bottom of the array as on the left:

```
i = n;
while ((a[i]>=x) && (i>=1)) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

```
i = n;
a[0] = - MAXINT;
while (a[i] >= x) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

or, we can make sure that fake element `a[0]` is smaller than anything it will encounter as on the right. Proper use of sentinels, and making sure that your array is a little larger than it presumably needs to be, can help avoid many boundary errors.

- *Multidimensional Arrays* — Rectangular grid structures such as chessboards and images comes to mind first when thinking about two-dimensional arrays, but more generally they can be used to group together *homogeneous* data records. For example, an array of n points in the $x - y$ plane can be thought of as an $n \times 2$ array, where the second argument (0 or 1) of $A[i][j]$ dictates whether we are referring to the x or y coordinate of the point.

- *Records* — These are used to group together *heterogeneous* data records. For example, an array of people-records can lump together people's names, ID numbers, heights, and weights into a simple package. Records are important for conceptual clarity in large programs, but such fields can often be harmlessly represented using separate arrays in programs of modest size.

  Whether it is better to use records or multidimensional arrays in a problem is not always a clear-cut decision. Think of the problem of representing points in the $x - y$ plane discussed above. The obvious representation would be a record or structure such as:

```
struct point {
     int x, y;
};
```

instead of as a two-element array. A big plus for records is that the notation `p.y` and `p.y` are more akin to our natural notation for working with points. However, a

disadvantage of the record representation is that you cannot loop over individual variables as you can elements in an array.

Suppose you wanted to change a geometric program to work with three-dimensional points instead of two, or even in arbitrary dimensions. Sure you can easily add extra fields to the record, but every place where you did calculations on x and y you now must replicate them for z. But by using the array representation, changing distance computations from two to three dimensions can be as simple as changing a constant:

```
typedef int point[DIMENSION];

double distance(point a, point b)
{
        int i;
        double d=0.0;

        for (i=0; i<DIMENSION; i++)
                d = d + (a[i]-b[i]) * (a[i]-b[i]);

        return( sqrt(d) );
}
```

In Chapter 2, we will review more advanced data *structures* which can be built from these basic primitives. These will let us work with higher levels of abstraction, but don't be afraid to use simple technology when it suffices for the job.

1.5 About the Problems

Each chapter of this book ends with an appropriate set of programming challenge problems. These challenges have been carefully selected from over 1,000 such problems collected at the Universidad de Valladolid website. We aimed for clear, clever problems with varying degrees of difficulty. We especially looked for that spark of insight which turns a problem into a challenge.

The description of each selected problem has been edited for correctness and readability. We have tried to preserve the local color and flavor of each original problem while making the language reasonably consistent. The identification number for each problem on both judging sites is provided. These numbers are important for successful submission. The first number for each pair is associated with *http://www.programming-challenges.com*; the second number works on *http://online-judge.uva.es*.

To give some idea as to the relative difficulty of the problems, each has been annotated in three different ways. First, each problem has been given a grade of `A`, `B`, or `C`, reflecting how many correct solutions the judge has seen over the years. `A` problems are presumably easier to solve or somehow more attractive than `B` problems. Second, the frequency that submitted solutions for each problem are accepted by the judge is graded as `high`,

`average`, or `low`. Low percentages may indicate particularly finicky judges, or perhaps problems that require more subtlety than is initially apparent. Or they may just reflect bugs in test cases which have since been fixed. Therefore, do not obsess too much about these ratings. Finally, we give a subjective rating (from 1 to 4) of the academic level required to solve the problem. Higher numbers indicate more sophisticated problems.

Good luck, and happy hacking!

1.6 Problems

1.6.1 The $3n + 1$ Problem

PC/UVa IDs: 110101/100, **Popularity:** A, **Success rate:** low **Level:** 1

Consider the following algorithm to generate a sequence of numbers. Start with an integer n. If n is even, divide by 2. If n is odd, multiply by 3 and add 1. Repeat this process with the new value of n, terminating when $n = 1$. For example, the following sequence of numbers will be generated for $n = 22$:

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

It is *conjectured* (but not yet proven) that this algorithm will terminate at $n = 1$ for every integer n. Still, the conjecture holds for all integers up to at least $1,000,000$.

For an input n, the *cycle-length* of n is the number of numbers generated up to and *including* the 1. In the example above, the cycle length of 22 is 16. Given any two numbers i and j, you are to determine the maximum cycle length over all numbers between i and j, *including* both endpoints.

Input

The input will consist of a series of pairs of integers i and j, one pair of integers per line. All integers will be less than 1,000,000 and greater than 0.

Output

For each pair of input integers i and j, output i, j in the same order in which they appeared in the input and then the maximum cycle length for integers between and including i and j. These three numbers should be separated by one space, with all three numbers on one line and with one line of output for each line of input.

Sample Input

```
1 10
100 200
201 210
900 1000
```

Sample Output

```
1 10 20
100 200 125
201 210 89
900 1000 174
```

1.6.2 Minesweeper

PC/UVa IDs: 110102/10189, **Popularity:** A, **Success rate:** high **Level:** 1

Have you ever played Minesweeper? This cute little game comes with a certain operating system whose name we can't remember. The goal of the game is to find where all the mines are located within a $M \times N$ field.

The game shows a number in a square which tells you how many mines there are adjacent to that square. Each square has at most eight adjacent squares. The 4×4 field on the left contains two mines, each represented by a "`*`" character. If we represent the same field by the hint numbers described above, we end up with the field on the right:

```
*...        *100
....        2210
.*..        1*10
....        1110
```

Input

The input will consist of an arbitrary number of fields. The first line of each field contains two integers n and m ($0 < n, m \leq 100$) which stand for the number of lines and columns of the field, respectively. Each of the next n lines contains exactly m characters, representing the field.

Safe squares are denoted by "`.`" and mine squares by "`*`," both without the quotes. The first field line where $n = m = 0$ represents the end of input and should not be processed.

Output

For each field, print the message `Field #`x`:` on a line alone, where x stands for the number of the field starting from 1. The next n lines should contain the field with the "`.`" characters replaced by the number of mines adjacent to that square. There must be an empty line between field outputs.

Sample Input

```
4 4
*...
....
.*..
....
3 5
**...
.....
.*...
0 0
```

Sample Output

```
Field #1:
*100
2210
1*10
1110

Field #2:
**100
33200
1*100
```

1.6.3 The Trip

PC/UVa IDs: 110103/10137, **Popularity:** B, **Success rate:** average **Level:** 1

A group of students are members of a club that travels annually to different locations. Their destinations in the past have included Indianapolis, Phoenix, Nashville, Philadelphia, San Jose, and Atlanta. This spring they are planning a trip to Eindhoven.

The group agrees in advance to share expenses equally, but it is not practical to share every expense as it occurs. Thus individuals in the group pay for particular things, such as meals, hotels, taxi rides, and plane tickets. After the trip, each student's expenses are tallied and money is exchanged so that the net cost to each is the same, to within one cent. In the past, this money exchange has been tedious and time consuming. Your job is to compute, from a list of expenses, the minimum amount of money that must change hands in order to equalize (within one cent) all the students' costs.

Input

Standard input will contain the information for several trips. Each trip consists of a line containing a positive integer n denoting the number of students on the trip. This is followed by n lines of input, each containing the amount spent by a student in dollars and cents. There are no more than 1000 students and no student spent more than \$10,000.00. A single line containing 0 follows the information for the last trip.

Output

For each trip, output a line stating the total amount of money, in dollars and cents, that must be exchanged to equalize the students' costs.

Sample Input

```
3
10.00
20.00
30.00
4
15.00
15.01
3.00
3.01
0
```

Sample Output

```
$10.00
$11.99
```

1.6.4 LCD Display

PC/UVa IDs: 110104/706, **Popularity:** A, **Success rate:** average **Level:** 1

A friend of yours has just bought a new computer. Before this, the most powerful machine he ever used was a pocket calculator. He is a little disappointed because he liked the LCD display of his calculator more than the screen on his new computer! To make him happy, write a program that prints numbers in LCD display style.

Input

The input file contains several lines, one for each number to be displayed. Each line contains integers s and n, where n is the number to be displayed ($0 \leq n \leq 99,999,999$) and s is the size in which it shall be displayed ($1 \leq s \leq 10$). The input will be terminated by a line containing two zeros, which should not be processed.

Output

Print the numbers specified in the input file in an LCD display-style using s "-" signs for the horizontal segments and s "|" signs for the vertical ones. Each digit occupies exactly $s + 2$ columns and $2s + 3$ rows. Be sure to fill all the white space occupied by the digits with blanks, including the last digit. There must be exactly one column of blanks between two digits.

Output a blank line after each number. You will find an example of each digit in the sample output below.

Sample Input

```
2 12345
3 67890
0 0
```

Sample Output

```
    --   --         --
  |    |    | |  | |
  |    |    | |  | |
     --   --   --   --
  | |       |    |    |
  | |       |    |    |
     --   --         --

 ---   ---   ---   ---   ---
|         | |   | |   | |   |
|         | |   | |   | |   |
|         | |   | |   | |   |
 ---         ---   ---
|   |     | |   |     | |   |
|   |     | |   |     | |   |
|   |     | |   |     | |   |
 ---         ---   ---   ---
```

1.6.5 Graphical Editor

PC/UVa IDs: 110105/10267, **Popularity:** B, **Success rate:** low **Level:** 1

Graphical editors such as Photoshop allow us to alter bit-mapped images in the same way that text editors allow us to modify documents. Images are represented as an $M \times N$ array of pixels, where each pixel has a given color.

Your task is to write a program which simulates a simple interactive graphical editor.

Input

The input consists of a sequence of editor commands, one per line. Each command is represented by one capital letter placed as the first character of the line. If the command needs parameters, they will be given on the same line separated by spaces.

Pixel coordinates are represented by two integers, a column number between $1 \ldots M$ and a row number between $1 \ldots N$, where $1 \leq M, N \leq 250$. The origin sits in the upper-left corner of the table. Colors are specified by capital letters.

The editor accepts the following commands:

I M N	Create a new $M \times N$ image with all pixels initially colored white (`O`).
C	Clear the table by setting all pixels white (`O`). The size remains unchanged.
L X Y C	Colors the pixel (X, Y) in color (`C`).
V X Y1 Y2 C	Draw a vertical segment of color (`C`) in column X, between the rows $Y1$ and $Y2$ inclusive.
H X1 X2 Y C	Draw a horizontal segment of color (`C`) in the row Y, between the columns $X1$ and $X2$ inclusive.
K X1 Y1 X2 Y2 C	Draw a filled rectangle of color `C`, where $(X1, Y1)$ is the upper-left and $(X2, Y2)$ the lower right corner.
F X Y C	Fill the region R with the color `C`, where R is defined as follows. Pixel (X, Y) belongs to R. Any other pixel which is the same color as pixel (X, Y) and shares a common side with any pixel in R also belongs to this region.
S Name	Write the file name in MSDOS 8.3 format followed by the contents of the current image.
X	Terminate the session.

Output

On every command `S NAME`, print the filename $NAME$ and contents of the current image. Each row is represented by the color contents of each pixel. See the sample output.

Ignore the entire line of any command defined by a character other than `I, C, L, V, H, K, F, S,` or `X`, and pass on to the next command. In case of other errors, the program behavior is unpredictable.

Sample Input

```
I 5 6
L 2 3 A
S one.bmp
G 2 3 J
F 3 3 J
V 2 3 4 W
H 3 4 2 Z
S two.bmp
X
```

Sample Output

```
one.bmp
OOOOO
OOOOO
OAOOO
OOOOO
OOOOO
OOOOO
two.bmp
JJJJJ
JJZZJ
JWJJJ
JWJJJ
JJJJJ
JJJJJ
```

1.6.6 Interpreter

PC/UVa IDs: 110106/10033, **Popularity:** B, **Success rate:** low **Level:** 2

A certain computer has ten registers and 1,000 words of RAM. Each register or RAM location holds a three-digit integer between 0 and 999. Instructions are encoded as three-digit integers and stored in RAM. The encodings are as follows:

`100`	means *halt*
`2`*dn*	means *set register d to n (between 0 and 9)*
`3`*dn*	means *add n to register d*
`4`*dn*	means *multiply register d by n*
`5`*ds*	means *set register d to the value of register s*
`6`*ds*	means *add the value of register s to register d*
`7`*ds*	means *multiply register d by the value of register s*
`8`*da*	means *set register d to the value in RAM whose address is in register a*
`9`*sa*	means *set the value in RAM whose address is in register a to that of register s*
`0`*ds*	means *goto the location in register d unless register s contains 0*

All registers initially contain 000. The initial content of the RAM is read from standard input. The first instruction to be executed is at RAM address 0. All results are reduced modulo 1,000.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, each described as below. This is followed by a blank line, and there will be a blank line between each two consecutive inputs.

Each input case consists of up to 1,000 three-digit unsigned integers, representing the contents of consecutive RAM locations starting at 0. Unspecified RAM locations are initialized to 000.

Output

The output of each test case is a single integer: the number of instructions executed up to and including the *halt* instruction. You may assume that the program does halt. Separate the output of two consecutive cases by a blank line.

Sample Input

```
1

299
492
495
```

```
399
492
495
399
283
279
689
078
100
000
000
000
```

Sample Output

```
16
```

1.6.7 Check the Check

PC/UVa IDs: 110107/10196, **Popularity:** B, **Success rate:** average **Level:** 1

Your task is to write a program that reads a chessboard configuration and identifies whether a king is under attack (in check). A king is in check if it is on square which can be taken by the opponent on his next move.

White pieces will be represented by uppercase letters, and black pieces by lowercase letters. The white side will always be on the bottom of the board, with the black side always on the top.

For those unfamiliar with chess, here are the movements of each piece:

Pawn (`p` or `P`): can only move straight ahead, one square at a time. However, it takes pieces diagonally, and that is what concerns you in this problem.

Knight (`n` or `N`) : has an L-shaped movement shown below. It is the only piece that can jump over other pieces.

Bishop (`b` or `B`) : can move any number of squares diagonally, either forward or backward.

Rook (`r` or `R`) : can move any number of squares vertically or horizontally, either forward or backward.

Queen (`q` or `Q`) : can move any number of squares in any direction (diagonally, horizontally, or vertically) either forward or backward.

King (`k` or `K`) : can move one square at a time in any direction (diagonally, horizontally, or vertically) either forward or backward.

Movement examples are shown below, where "`*`" indicates the positions where the piece can capture another piece:

```
Pawn        Rook        Bishop      Queen       King        Knight
........    ...*....    .......*    ...*...*    ........    ........
........    ...*....    *.....*.    *..*..*.    ........    ........
........    ...*....    .*...*..    .*.*.*..    ........    ..*.*...
........    ...*....    ..*.*...    ..***...    ..***...    .*...*..
...p....    ***r****    ...b....    ***q****    ..*k*...    ...n....
..*.*...    ...*....    ..*.*...    ..***...    ..***...    .*...*..
........    ...*....    .*...*..    .*.*.*..    ........    ..*.*...
........    ...*....    *.....*.    *..*..*.    ........    ........
```

Remember that the knight is the only piece that can jump over other pieces. The pawn movement will depend on its side. If it is a black pawn, it can only move one square diagonally down the board. If it is a white pawn, it can only move one square diagonally up the board. The example above is a black pawn, described by a lowercase "`p`". We use "*move*" to indicate the squares where the pawn can capture another piece.

Input

There will be an arbitrary number of board configurations in the input, each consisting of eight lines of eight characters each. A "." denotes an empty square, while upper- and lowercase letters represent the pieces as defined above. There will be no invalid characters and no configurations where both kings are in check. You must read until you find an empty board consisting only of "." characters, which should not be processed. There will be an empty line between each pair of board configurations. All boards, except for the empty one, will contain exactly one white king and one black king.

Output

For each board configuration read you must output one of the following answers:
`Game #`d`: white king is in check.`
`Game #`d`: black king is in check.`
`Game #`d`: no king is in check.`
where d stands for the game number starting from 1.

Sample Input

```
..k.....
ppp.pppp
........
.R...B..
........
........
PPPPPPPP
K.......

rnbqk.nr
ppp..ppp
....p...
...p....
.bPP....
.....N..
PP..PPPP
RNBQKB.R

........
........
........
........
........
........
........
........
```

Sample Output

```
Game #1: black king is in check.
Game #2: white king is in check.
```

1.6.8 Australian Voting

PC/UVa IDs: 110108/10142, **Popularity:** B, **Success rate:** low **Level:** 1

Australian ballots require that voters rank all the candidates in order of choice. Initially only the first choices are counted, and if one candidate receives more than 50% of the vote then that candidate is elected. However, if no candidate receives more than 50%, all candidates tied for the lowest number of votes are eliminated. Ballots ranking these candidates first are recounted in favor of their highest-ranked non-eliminated candidate. This process of eliminating the weakest candidates and counting their ballots in favor of the preferred non-eliminated candidate continues until one candidate receives more than 50% of the vote, or until all remaining candidates are tied.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases following, each as described below. This line is followed by a blank line. There is also a blank line between two consecutive inputs.

The first line of each case is an integer $n \leq 20$ indicating the number of candidates. The next n lines consist of the names of the candidates in order, each up to 80 characters in length and containing any printable characters. Up to 1,000 lines follow, each containing the contents of a ballot. Each ballot contains the numbers from 1 to n in some order. The first number indicates the candidate of first choice; the second number indicates candidate of second choice, and so on.

Output

The output of each test case consists of either a single line containing the name of the winner or several lines containing the names of all candidates who are tied. The output of each two consecutive cases are separated by a blank line.

Sample Input

```
1

3
John Doe
Jane Smith
Jane Austen
1 2 3
2 1 3
2 3 1
1 2 3
3 1 2
```

Sample Output

```
John Doe
```

1.7 Hints

1.3 Who should get the extra pennies if the total does not divide evenly?

1.5 How do we best handle the fill command? Is it easier to keep separate copies of the old and new image?

1.8 Notes

1.1 The $3n + 1$ (or Collatz) problem remains open to this day. See Lagarias [Lag85] for an excellent mathematical survey. An international conference on the Collatz problem was held in 1999; see *http://www.math.grinnell.edu/~chamberl/conf.html* for the online proceedings.

1.2 The Minesweeper consistency problem takes as input an $n \times n$ rectangular grid with the squares labeled by numbers 0 to 8, mines, or left blank, and asks: is there a configuration of mines in the grid that result in the given pattern of symbols, according to the usual Minesweeper rules? The Clay Institute of Mathematics (*http://www.claymath.org/*) offers a $1,000,000 prize for an efficient algorithm which solves this problem.

But don't get too excited! The Minesweeper consistency problem has been proven NP-complete [Kay00], meaning that no efficient algorithm for it can exist without revolutionizing our understanding of computation. See [GJ79] for a thorough discussion of NP-completeness.

1.6 Software-interpreted virtual machines are the key to the portability of languages such as Java. It is an interesting project to write a simulator for the machine language of an old, obsolete, but simple computer such as the PDP-8. Today's hardware is so much faster that your virtual PDP-8 will significantly outrun the original machine!

1.7 Once you have written a legal move generator (the essence of this problem) you are not very far from building your own chess-playing program! See [New96, Sch97] for stories of how computer chess and checkers programs work and beat the human World Champions at their own games.

1.8 The mathematics of voting systems is a fascinating subject. Arrow's *impossibility theorem* states that no voting system can simultaneously satisfy all of five obviously desirable properties. See [COM94] for an interesting discussion of the mathematics of social choice.